RuSMT: An Executable Semantics as Conformance Oracle and Test Suite Synthesizer

Mehrad Haghshenas
m3haghsh@uwaterloo.ca
University of Waterloo
Waterloo, Ontario, Canada

Meng Xu
meng.xu.cs@uwaterloo.ca
University of Waterloo
Waterloo, Ontario, Canada

Abstract

A language ecosystem ships a prose specification, an implementation, and a conformance suite, and these three are supposed to agree. Written separately they drift, and the cases a specification cares about most, namely the malformed input, the out-of-range literal, the duplicate key, and the division by zero, are the ones a fuzzer rarely reaches and a hand-written suite under-populates. This paper presents RuSMT, a framework in which the author writes the semantics once, as an ordinary recursive Rust program in a small DSL whose types denote Z3's theories, and marks the branches to be covered with a named marker such as `Path::named("division_by_zero")`. That one source is read two ways: compiled by `rustc` it runs as the reference oracle, and transpiled to SMT-LIB it becomes one reachability query per marker. The product of the framework is a conformance suite, one input per marker, which we then run against independent implementations of the language.

The hard part is not writing the semantics but finding the inputs. On a TOML 1.1.0 parser with 182 markers, Z3 on its own produced no witness at any budget we could afford, and we explain why: the transpiled parser functions are macros that Z3 expands before it can know which branch is taken, so a symbolic document keeps both arms of every nested conditional alive. RuSMT therefore runs a two-stage loop. Stage 1 asks Z3 to synthesize the input directly, which is what happens for IMP. Stage 2 is entered only on a timeout or `unknown`: a language model reads the emitted SMT-LIB and Z3's previous verdicts and proposes one concrete candidate, which is pinned into the *unmodified* query as a single equality. Only a `sat` answer from Z3 makes it a witness, so the model shapes the search but never decides acceptance. In the reported run this produced 131 of 182 TOML markers and 2 of 2 IMP markers; the 51 misses are named. Running the generated TOML suite against four independent parsers exposed 15 accept/reject divergences: three are parser bugs, and twelve are places where our own reference turned out to be stricter than TOML requires. We report the numbers, the misses, and the reasons for both.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; **Formal software verification**; • **Theory of computation** → *Automated reasoning*.

Keywords: executable semantics, conformance testing, bounded program synthesis, SMT, Z3, solver-aided programming, Rust, large language models

1 Introduction

A language ecosystem typically carries three artifacts that are meant to say the same thing: a prose *specification*, an *implementation* that is supposed to adhere to it, and a *conformance suite* that is supposed to check the adherence. Written separately, they drift. Fuzzing covers the bulk of the input space but tends to miss the narrow error conditions a specification cares about most, and a hand-written suite forces its author both to decide which conditions are interesting and to construct the inputs that exercise them. RuSMT keeps the first of those two jobs with the author, who expresses the interesting conditions as markers inside the semantics, and hands the second to a solver. Accordingly, when the specification changes, the suite is *regenerated* rather than re-authored.

Why error cases. Markers target what we will call a language's *negative space*: its *syntactic* errors, that is, ill-formed input the grammar rejects, and its *semantic* errors, that is, well-formed input that violates the static or dynamic semantics, such as an out-of-range value, a duplicate key, a type error, or a division by zero. This is the textbook front-end/analysis boundary [16], and our focus on it is empirically grounded rather than a convenience of what the solver happens to find easy. Error-handling code is where defects concentrate and where testing is thinnest: a majority of *catastrophic* distributed-systems outages trace back to incorrect handling of errors, often in code a trivial test would have caught [30], and error paths are routinely dropped in Linux file systems [8]. It is also where implementations *diverge*. No two JSON parsers agree across malformed documents [21], parser and validator discrepancies on ill-formed input are a documented source of security vulnerabilities [4, 20], and it is exactly what a conformance suite asserts normatively [6]. RuSMT therefore does not fuzz values broadly. It synthesizes inputs that reach the specification's own error branches, and then differentially tests how independent implementations treat those inputs [13].

One source, two readings. RuSMT collapses two of the three artifacts into one. The author writes the executable

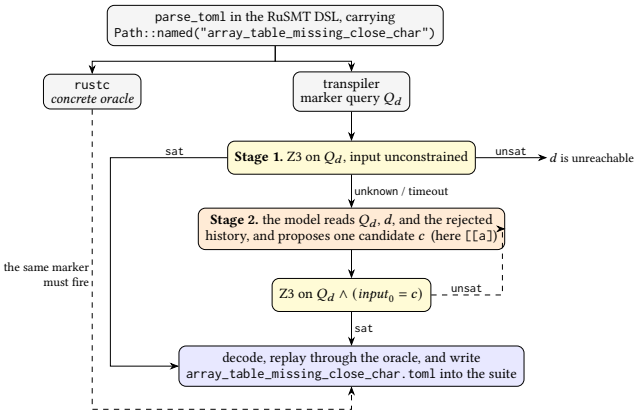

Figure 1. One executable semantics, two readings, and the loop that turns one named marker into one conformance test. The marker shown is the running example of §6.2 and §7.

semantics *once*, in a restricted dialect of Rust whose standard library types denote Z3's mathematical theories. That single source is then given two interpretations. Run as ordinary Rust it is the concrete conformance *oracle*; lifted to SMT-LIB it is the source of a solver query that *synthesizes* inputs reaching designated points. The author designates a point by writing a named marker, for instance `Path::named("division_by_zero")`, on a branch a total evaluator has to have anyway.

The loop, and one marker through it. Figure 1 is the whole method, and we will thread a single marker through it for the rest of the paper. The TOML reference semantics raises `array_table_missing_close_char` when an array-of-tables header is not closed. RuSMT emits one query Q_d for that marker, asserting that the entry function returns an error carrying that marker's id. Stage 1 hands Q_d to Z3 with the document left unconstrained. For this marker, as for every other TOML marker, Z3 returns nothing useful, so Stage 2 begins: a language model is shown the emitted SMT-LIB, the marker's name, and whatever candidates have already been rejected, and it answers with one concrete document, here `[[a]]`. The pipeline appends a single equality $input_0$ = `[[a]]` to the *unmodified* Q_d and asks Z3 to decide it. Z3 answered `sat` in 1.24 s, which by the strengthening lemma (§5) is a statement about Q_d itself and not about the strengthened query. The decoded document is then re-executed through the concrete Rust semantics, where it must fire the *same* marker. It does, so `[[a]]` enters the suite. Later, in §7, one of the four TOML parsers we tested accepts that document as valid.

Two things about this loop are worth stating up front, because they are what the rest of the paper defends. First, the order matters. Z3 attempts every marker unaided before any model is consulted, and it is the only component that can answer `unsat`, which for an unconstrained query is a statement that *no* input reaches the marker. Second, the model never produces a witness. Its output is a candidate, and a candidate only becomes a test by surviving Z3 and then replay. In other words, the model is a search heuristic that we allow to be wrong, and in the reported run it was wrong most of the time.

Research questions. *RQ1:* Can a direct-style Rust interpreter, written in a small SMT-denoting DSL, serve simultaneously as a conformance oracle and as a marker-directed test generator? *RQ2:* What discipline is sufficient for accepting a generated test without a human looking at it? *RQ3:* When Z3 alone fails on a realistic parser, how much coverage does the Stage-2 loop recover, and where does it still fail? *RQ4:* Does the Z3 acceptance step reject model proposals often enough to be doing real work? *RQ5:* Run against independent implementations, what does the generated suite expose?

Contributions. This is a tool contribution. We are not proposing a new solver, a new synthesis algorithm, or a completeness theorem. **(1)** A Rust-native framework in which an ordinary recursive interpreter yields two artifacts from one source: compiled, the conformance *oracle*; lowered to SMT-LIB, a marker-directed *test generator*. There is no separate grammar to write and no hand-written SMT (§3–§4). **(2)** A Z3-first, model-second pipeline in which the model sees only the emitted SMT-LIB and the solver's feedback, never the Rust source (§4). **(3)** An acceptance discipline that needs no human inspection, resting on the strengthening lemma rather than on trust in the model (§5). **(4)** Measured conformance suites for TOML 1.1.0 and IMP, covering 131/182 and 2/2 markers respectively, with every miss named (§7). **(5)** A downstream differential comparison of four production TOML parsers on the generated suite, yielding 15 accept/reject divergences (§7).

Structure. The paper is organized as follows. Section 2 gives the background on SMT as a reasoning backend and on executable semantics. Section 3 presents the RuSMT DSL and argues why each restriction earns its place. Section 4 describes the compilation pipeline and the two-stage loop, including the encoding choices that were made for the solver's benefit. Section 5 states what RuSMT asks the reader to trust and what it proves; we are explicit here because the framework is a trusted computing base in the same sense a proof assistant's kernel is. Section 6 introduces the two case studies, and Section 7 reports the run and answers the research questions. Sections 8 and 9 cover related work and the limitations we are aware of, and Section 10 concludes.

2 Background

SMT as the reasoning backend. RuSMT uses SMT to ask exactly one operational question: *does there exist an input*

such that this executable semantics reaches marker d? The DSL therefore maps Rust expressions onto a fixed vocabulary of Z3 theories [5]: booleans, integers and reals, bit-vectors, strings, sequences, sets, arrays, algebraic datatypes, and recursive functions, all encoded in SMT-LIB 2.6 [2]. For each marker the backend emits a formula of the form

$$\exists x.\ entry(x) = Err(marker\_id(d)),$$

where x is the object-language program or input text. On `sat`, Z3 returns a model for x; on `unsat`, no such x exists *for that encoding and bound*; on `unknown` or timeout, we learn only that this backend did not finish.

The difficulty is that these formulas fall outside any decidable quantifier-free fragment. A recursive interpreter becomes `define-funs-rec`, which Z3 handles by unfolding and heuristics, and a parser additionally combines sequences, arithmetic, bit-vectors, recursive datatypes, and the occasional quantifier discharged by incomplete instantiation. We therefore treat Z3 as sound for the verdicts it returns and as an incomplete search procedure otherwise: an `unknown` or a timeout costs us coverage and nothing else. It is worth noting already that this same split is what makes Stage 2 useful. Searching for an unconstrained input can be intractable while *deciding* a fully determined candidate is easy, because a concrete input lets the simplifier fold each conditional as the definitions are inlined.

Executable semantics and synthesis. A *semantics* fixes the meaning of a language, and it is *executable* when that definition is itself a runnable artifact, that is, a reference interpreter written *to be* the specification rather than merely an implementation that happens to run programs. K [18], PLT Redex [7], and Ott/Lem [14, 22] write that definition as rewrite or inference rules and derive a tool suite from it. RuSMT instead takes the semantics to be an *ordinary recursive function*, so that the same source executes concretely and compiles symbolically. Compared with fuzzers such as Csmith [29], the target is not coverage in general but a named specification obligation. Compared with SyGuS and SemGuS [1, 9], the user does not author a separate grammar or a set of constrained Horn clauses; the unknown is simply an input to the interpreter the user has already written.

3 The RuSMT DSL

A RuSMT specification is an ordinary Rust program written once and read twice: compiled by `rustc` it *runs*, and lifted by the transpiler it *denotes* an SMT formula. The design principle throughout is that these two readings must agree by construction, and consequently every construct admitted into the DSL is one with a mathematical meaning the solver can express. That single requirement dictates the whole subset: RuSMT is a deliberately *functional* and *total* fragment of Rust.

A functional subset. The DSL keeps immutable `let` bindings, `if` with a mandatory `else`, `struct` and `enum` with `match`, and mutually recursive calls. It forbids loops, mutation, and references. These exclusions are not arbitrary; each one buys a property the lift depends on. Forbidding mutation and references means a value never aliases and never changes under another name, so a `let` is mathematical substitution and the translation is referentially transparent. This is the same discipline that lets a big-step operational semantics be read as a system of equations. Forbidding loops leaves recursion and bounded quantifiers as the only means of iteration, which matches how a solver expresses repetition, namely as recursive definitions and quantified formulas rather than as mutable state evolving over time. The payoff is that the interpreter ends up reading like the inference rules it formalizes. Winskel's big-step rules for IMP (§6.1) transcribe almost line for line, and there is no gap between "the specification" and "the code that runs."

Totality makes error cases first-class. The subset is also *total* and exception-free: no `panic!`, no `?`, no `.unwrap()`, and no early return. A partial function has no total SMT reading, so any divergence from the happy path must be expressed as a returned *value*, for example a `ParseResult::Err` carrying a `Path`, rather than as a thrown exception. This is precisely what the negative-space thesis of §1 needs, and we get it for free: every error condition is, by construction, an explicit branch returning an error value, which is to say a place where a marker can sit, on code that has to exist anyway.

Separation of concerns. Two layers are kept strictly apart. The *standard library* is a fixed vocabulary denoting *Z3's* theories and nothing else, whereas the *interpreter* encodes a *target* language's semantics on top of that vocabulary. A target's idiosyncrasies, such as two's-complement wraparound, a parser's error conditions, or an uninitialized-variable rule, live in the interpreter as explicit guards and never inside the stdlib. This keeps the per-construct obligation of §5 small, fixed, and language-independent, so that it is argued once for the stdlib rather than re-argued for every case study.

Mechanical enforcement. Two attribute macros enforce the subset at compile time, so an out-of-subset specification fails to build rather than mistranslating silently. `#[smt_type]` admits only `structs` and `enums` whose every value is `Copy`, so that nothing aliases, and `#[smt_fn]` rejects `const`, `async`, `unsafe` and variadic functions, method receivers, and any body beyond `let` bindings and a trailing expression.

Listing 1 shows the whole discipline in one IMP fragment, namely the division guard of the arithmetic evaluator. It is ordinary recursive Rust, and the only synthesis-specific token is `Path::named`, sitting on an error branch that a total evaluator must have anyway. The primitive `bv_div` already

agrees with Z3 on every nonzero divisor, so the interpreter guards only the diverging case and falls through otherwise, which is the guard pattern of §5. From this single source RuSMT synthesizes `A := (0 / 0)`, and replaying it through the same `eval_aexp` reaches the marker.

```
if *new_r.eq(I64::from(0)) {
    ArithmeticResult::Err(Path::named("division_by_zero"))
} else {
    ArithmeticResult::Ok(new_l.bv_div(new_r))
}
```

Listing 1. The division guard of IMP's evaluator (operands already evaluated to `new_l` and `new_r`; `Err` propagation elided).

Standard library: the theory vocabulary. The DSL's types denote Z3 sorts: `Boolean`, the unbounded `Integer`/`Real` (big integers and rationals), the bit-vectors `I32`/`I64`/`U32`/`U64` (the signed/unsigned split is a DSL-level reading of the same Z3 sort), IEEE `F32`/`F64`, `String`, `Seq<T>`, `Set<T>`, and `Array<K,V>`. As stated above, these denote *Z3's* theories, not Rust's native semantics and not any target language's. Two helper types are frontend-only. `Cloak<T>` is a `Box`-like indirection that lets an author write self-referential ADTs, for example `Add(Cloak<Aexp>, Cloak<Aexp>)`, despite Rust's sized-type rule; the IR *erases* it entirely, lowering every `Cloak<T>` field to `T` and `shield`/`reveal` to the identity, since Z3 supports recursive datatypes natively.[1] `Path`, the marker type, is described next.

Markers via `Path`. A marker flags a program point the author wants covered, characteristically an error or specification-violation branch. What a marker carries is a `Path`, and its *identity* is what allows the marker to be matched across the two readings of the specification. An id drawn from a global counter would be order-dependent and therefore unmatchable against a replayed run. Instead, `Path::named("`*d*`")` derives its id as a fixed hash of the name *d*, and the *same* pure function serves the transpiler and the evaluator, so the id the query asserts and the id a re-executed witness carries are identical by construction. In the backend a `Path` is represented by that id as a plain `Integer`, with a sentinel for "no marker reached". This choice is deliberate rather than incidental: an integer equality survives the deep recursive unfolding of a lifted interpreter, whereas a set-membership test would force the solver to construct and probe a symbolic collection it cannot evaluate along those paths.

Quantifiers. `forall!`, `exists!`, and `choose!` exist in a *bounded* form over a collection, written `forall!(x in xs => p)`. `choose!` is Hilbert choice: concretely the first satisfying witness, and in SMT a Skolem function under the standard choose axiom. It is sound because nothing may depend on *which* witness is chosen, only on the fact that it satisfies the predicate.

[1] The one proviso is well-foundedness: a `Cloak<T>` must co-occur with a non-recursive variant, otherwise the lowered datatype is uninhabited.

4 From a marker to a conformance test

The same annotated source is compiled in two modes. In oracle mode `rustc` executes the interpreter on concrete inputs. In solver mode the transpiler parses the restricted subset, lowers it to a sort-checked IR by Hindley–Milner unification over SMT sorts, monomorphizes generic functions on demand, records each `Path::named` marker, and emits one query per marker. Every query is inspectable SMT-LIB2, which is what makes each result in this paper re-runnable with a bare `z3 -smt2` on a committed file.

4.1 Markers to queries

For a target marker with id $i = marker\_id(d)$, the query declares one constant per top-level parameter (`input_0`, …), applies the entry function, and asserts that the returned error value's carried `Path` *equals* i, guarded by the enum constructor test that selects the error variant, followed by `(check-sat)` and `(get-model)`.

A recursive interpreter is not a finite formula, and a `k=N` flag selects between two routes. With `N=0`, which is the default and what both case studies use, recursive strongly-connected components are emitted as Z3 `define-funs-rec` and the solver handles recursion natively. With $N \geq 1$ every recursive SCC is unrolled into $N{+}1$ depth-indexed copies $f_0, \ldots, f_N$, where f_d calls f_{d-1} for in-SCC callees, f_0 is a terminator, and external callers route through f_N. A `sat` model under depth-N unrolling is then a witness reachable in at most N recursive steps, which is the sense in which completeness is *bounded*, as in bounded model checking [3].

Unrolling cuts both ways and IMP shows both directions. At `k=1` the division-by-zero target returns `unsat`, yet the same marker is `sat` with a replay-accepted witness at `k=0`; a bounded `unsat` therefore means only "no witness within depth k" and never unreachability, which is why the pipeline treats `unsat` as a genuine verdict only at `k=0`. In the other direction, when the return type has no nullary variant the backend falls back to an *unconstrained* null sentinel at the cutoff, and the solver may instantiate it as the very error value the marker assertion demands, satisfying the assertion *through* the cutoff rather than along a real path. Replay rejects such candidates, so nothing unsound escapes, but the budget is wasted. The remedy we adopted is a nullary *bottom* variant that the concrete semantics never produces, after which the cutoff can no longer discharge an `Err`-marker assertion.

4.2 Stage 1, and why Stage 2 exists

Stage 1 hands the query to Z3 exactly as emitted, with the input unconstrained. This is the only configuration whose

`unsat` we are willing to read as unreachability, because only there does the absence of a model say something about the entire input space rather than about one candidate. For IMP, Stage 1 is the whole story.

For TOML it never once succeeded, and the way it failed is more informative than the failure itself. With a symbolic document Z3 does not report `unknown` after exhausting a budget; it produces no verdict at all within 300 s, and on partially-symbolic variants of the same query it terminates abnormally. Given a *determined* document the identical query decides in well under a second. The asymmetry is not really about the size of the search space. It is about macro expansion: the transpiled parser functions are non-recursive `define-fun` definitions, which Z3 substitutes during preprocessing, before it can know which branch will be taken. A concrete document lets the simplifier fold each conditional as it expands, while a symbolic one leaves both arms of every conditional alive and the expansion multiplies through the nesting. We tested the obvious alternative and it failed: emitting everything as `define-fun-rec`, so that unfolding becomes lazy, made matters *worse*, because the character-class predicates could then no longer fold either.

Stage 2 follows directly from that diagnosis. If the solver is cheap on determined inputs and hopeless on symbolic ones, then the productive move is to determine the input by some other means and let Z3 do what it is good at. The model receives the emitted SMT-LIB text, the marker name, and the candidates already rejected together with Z3's verdict on each, and returns one candidate. The pipeline appends a single equality, $input_0 = c$, to the *unmodified* query. A `sat` answer is a model of the original marker query; an `unsat` answer says only that this particular candidate does not reach this particular marker, and is fed back as the next round's context.

It is important to note what the model is not allowed to see. It runs in an empty working directory with file and shell tools disabled, so it cannot read `lang/src/toml/` and recover an answer from the Rust source. We checked this directly rather than assuming it: asked to quote a named function from the reference parser, the model replies that it has no file access. The claim that it is reasoning from the emitted SMT-LIB depends on this, so we state it as a configuration requirement and not as an aspiration.

A natural question, raised in review, is why the proposer should be a language model at all, rather than a bounded enumerator or a random generator. Nothing in the design requires a model: the proposer is an ordinary shell command that reads a prompt on standard input and writes a candidate on standard output, so a local model, an enumerator, or a fuzzer is a drop-in replacement, and, more importantly, the acceptance step is unchanged by the substitution, so the guarantee of §5 is unchanged too. Our reason for not using an undirected generator is an inference from the numbers in §7 rather than a separate experiment, and we flag it as such: of the proposals a *marker-targeted* generator made, Z3 rejected roughly five in six, and the dominant rejection reason was that an earlier error in the document shadowed the intended one. A generator with no notion of the marker faces a strictly harder version of that problem.

4.3 Engineering for Z3

Several encodings exist purely for the solver's benefit, and we list them because they were the bulk of the work.[2] A marker is an `Integer` id rather than a symbolic set membership test, because equality survives deep recursive unfolding. Recursive maps such as TOML tables are encoded as array-free inductive association lists; an `Array` whose range is a recursive datatype made even concrete-candidate validation return `unknown` in early TOML experiments. Recursion is offered either as native `define-funs-rec` or as fixed-depth unrolling, because neither dominates across the two case studies. Finally, the backend parses Z3's leading status line before trusting the process exit status, since Z3 can exit nonzero after a benign failure to print a model.

A `sat` model is rendered back to object-language source by a printer that extracts `input_0`, inlines `let`-shared subterms, maps mangled constructors back to AST nodes, canonicalizes solver-chosen identifiers, and decodes bit-vector literals as two's-complement signed integers. Canonicalizing identifiers is sound here because which marker fires does not depend on spelling. This rendering step is visible in IMP, where Z3 returns a `Com` AST that has to be printed as an IMP program; in TOML the top-level input is already a sequence of Unicode code points, so the decoded model is the document itself.

5 What RuSMT asks you to trust

We think it is more useful to state this plainly than to dress it up. RuSMT is a trusted computing base in the same sense that a proof assistant's kernel is one. Coq does not prove its own kernel; it asks you to accept it, and in exchange everything checked above it inherits that acceptance. RuSMT makes the same bargain at a much smaller scale. If you accept the framework, then you get conformance tests whose acceptance was decided by a solver rather than by a person reading them.

The trusted base. Four things are trusted. First, the RuSMT specification itself: the transcription of a prose standard into the DSL is done by a human and is not verified against the standard. Second, the standard library's correspondence with Z3, discussed below. Third, the transpiler that lifts the DSL to SMT-LIB. Fourth, Z3. The concrete parser and pretty-printer used for replay are trusted as well. A bug in any of these is a bug in our results, and we do not claim otherwise.

[2] Two of them, the marker representation and the array-free encoding, were changed only after the naïve version had been observed to fail.

The per-construct correspondence. Everything above rests on one obligation, stated for each standard-library primitive.

Definition 5.1 (Per-construct correspondence)**.** For every stdlib function f and every concrete input x satisfying f's documented preconditions,

$$rust\_impl(f, x) \;=\; z3\_eval\big(z3\_formula(f),\, x\big).$$

We *assume* this rather than prove it. What we do instead is make it auditable. Each stdlib method's documentation states the exact Z3 formula the backend emits for it, together with its preconditions and, where the two readings genuinely differ, the inputs on which they differ. Those divergences are real and we do not hide them; the documented ones are cases such as `String::to_int` on a non-numeric string, `String::from_int` on a negative integer, `replace_all` with an empty pattern, and `Array::select` on an absent key. In every such case the obligation is pushed onto the interpreter author, who must guard the input before the call. The TOML parser, for instance, checks each character with `is_hex_digit` and strips prefixes and underscores before anything reaches `Integer::from_hex_str`, so an invalid input becomes a named marker and never reaches the stdlib at all.

The consequence of this arrangement is a division of labour that we want to be explicit about. The stdlib is responsible for denoting Z3's theories faithfully within its stated preconditions, and the interpreter author is responsible for staying inside them and for bridging the gap between those theories and the target language. A primitive that matches the target exactly is used directly; a primitive that diverges only on specific inputs is *guarded* on those inputs, falling through to the primitive otherwise. Listing 1 has exactly this shape, and because concrete Rust and Z3 take the *same* branch on the same input, Definition 5.1's equality survives the conditional.

Single free input; fixed context. A load-bearing and easily overlooked point concerns *which* inputs are free.

Principle 1 (Free/fixed agreement)**.** The set of inputs treated as free by the SMT query must coincide with the set of inputs left free by the concrete oracle, and everything else must be fixed to the same value on both sides.

The IMP entry point `eval_com(c: Com)` fixes the empty store internally, so the program is the single free input. Had the store been left free, Z3 could have chosen a starting state the concrete run never uses, producing a "witness" that does not replay. TOML's `parse_toml` needs no such wrapper.

Why a `sat` answer is enough. This is the one place where we do prove something, and it is elementary. A Stage-2 candidate enters the query as an added conjunct, so the strengthened query is $Q_d \wedge (input_0 = c)$. Adding a conjunct cannot create a model that Q_d lacked; hence any model of the strengthened query is a model of Q_d. Consequently a `sat` verdict is a statement about the *original* query, which is the one encoding the marker the author named, and it remains so however the candidate was produced. This is why nothing about the model needs to be trusted, and it is also why we do not describe the model as "untrusted but checked": there is no checking step in which it could be trusted or not, only a logical fact about conjunction.

Replay, and what it is for.

Observation 1 (Marker-specific acceptance). A candidate is accepted for target d only if Z3 decides that the lifted semantics reach the *named* marker d, and the same candidate, re-executed through the concrete interpreter, fires the *same* d rather than merely some marker.

Replay re-parses and re-executes the decoded input under a wall-clock bound and checks membership of the *same* `marker_id(d)` that the query asserted, so a candidate that fires a *different* marker is rejected. We want to be precise about what this step does and does not do, because an earlier version of this work described it as a second gate on the model, which it is not. Replay is a check on the *lift*, not on the proposal. Since acceptance is already a logical consequence of `sat`, the only way an accepted candidate can be wrong is if Definition 5.1 fails on some construct the candidate exercises. Replay catches precisely that disagreement, and it fails safe: a faithfulness defect shows up as a *spurious rejection*, never as a false accept. It also rejects the depth-bounded spurious models described in §4, where Z3 reports `sat` with an empty reason-unknown.

Completeness: not claimed. We do not claim completeness in any general sense. Even when an input reaching a marker exists, Z3 may return `unknown` or time out, since the encoded formula can fall outside what the solver decides within its resource limits. At best we have *bounded* completeness relative to a structural or fuel bound, and even that is contingent on the solver.

6 Case studies

RuSMT is meant for authors of new executable semantics, so the two case studies were chosen to stress two different regimes rather than to maximize a benchmark count.

6.1 IMP: closing the loop end to end

IMP is the canonical small imperative "while" language of Winskel [26]. We chose it because its big-step semantics is familiar and recursive, yet small enough that native Z3 recursion ought to succeed if the pipeline is sound at all. Its grammar has arithmetic and boolean expressions, assignment, sequencing, conditionals, and while loops, and we added two dynamic error branches to make the conformance obligations explicit: `division_by_zero`, and `undefined_variable` for reading a variable from

the initially empty store. This study answers whether a direct-style interpreter can be lifted, solved, decoded, and replayed without any bounded unrolling and without any help from a proposer. Note that Z3 here synthesizes an AST rather than text, so the renderer of §4 is exercised as well.

6.2 TOML: a parser suite

TOML 1.1.0 [17] was chosen because it is large enough to defeat the solver while still having a precise public specification and several independent implementations to test against. The executable parser has a single free input, a sequence of Unicode code points, and 150 functions covering documents, key-value pairs, tables, arrays, inline tables, booleans, integers, floats, strings, and datetimes. Its 182 named markers cover syntactic errors, such as malformed keys, unterminated strings, invalid escapes, and missing array or table delimiters, as well as semantic violations, such as integer and float overflow, out-of-range date and time fields, duplicate keys, and redefined tables.

This is the case study that motivated the method, for the reasons given in §4. The witnesses that come out of the loop are small and readable, which is a property we did not design for but which matters downstream, since a minimal document isolates the rule it violates. Besides the running example `[[a]`, the run produced `a=1979-13-01` for the month-range marker and `a=[` for an unterminated array. We do not claim a verified or complete TOML formalisation; we claim a formalisation with 182 named obligations, and a suite generated from it.

7 Evaluation

Setup. All measurements were taken on an Apple M1 running macOS 26.5 with Z3 4.15.4, driven as a subprocess over SMT-LIB2. The IMP suite was generated by `cargo run -p rusmt-smt-derive – imp eval_com`, and the TOML suite by the same command with `toml parse_toml`, run through a resumable per-marker sweep driver with six workers so that a stopped run does not repeat model calls it has already paid for. The Stage-2 proposer was `gpt-5.4-mini` with `RUSMT_ROUNDS=9`, `RUSMT_WITNESSES=1`, and a 15 s Z3 budget for each pinned candidate. Stage 1 was given a deliberately small budget of 2 s per TOML marker, for the reason given in §4: separate probing had already established that a symbolic TOML input yields no verdict even at 300 s, so a larger Stage-1 budget buys nothing and is subtracted from rounds that do finish. Every model exchange is recorded and keyed by a content hash of the prompt, so the reported run replays without model access. Because the model is nondeterministic, re-running it from scratch is not guaranteed to regenerate an identical suite; the scientific object here is the recorded run together with the suite it produced, and not a claim of determinism.

Table 1. Synthesis and conformance results. The proposal audit covers the 156 markers whose per-round histories were recorded; see the text.

Check	Setting	Result
IMP Stage 1	$k = 0$	2/2 markers; ASTs rendered to IMP programs
TOML Stage 1	$k = 0$, 2 s	0/182 markers; no verdict at 300 s either
TOML Stage 2	`gpt-5.4-mini`, 9 rounds	131/182 markers; 51 misses, named
Proposal audit	649 decided proposals	105 accepted, 537 rejected `unsat`, 7 undecided
Conformance diff	131 tests × 4 parsers	15 accept/reject divergences

RQ1: one semantics as both oracle and generator. This is positive for both case studies, and it is the least surprising of the results. One Rust source served as the conformance oracle and as the source of every generated test; no separate grammar, model, or SMT file was written by hand. For IMP, under native recursion Z3 returned a model for both markers and both witnesses replayed. Z3 assembled them from the datatype constructors alone, after which the renderer printed `A := (0 / 0)` for `division_by_zero` and `v0 := v0` for `undefined_variable`. We do not run IMP through the differential harness, since it is a small end-to-end demonstration and not a language with competing production parsers.

RQ3: how much the loop recovers on TOML. We take RQ3 before RQ2 because the coverage numbers give the later questions their context. Lifting the TOML parser produces SMT that Z3 accepts without frontend error, but Stage 1 produced no witness for any of the 182 markers, in the manner described in §4. Stage 2 produced 131 accepted documents, so coverage went from 0/182 to 131/182.

The 51 misses are not spread uniformly, and their distribution is the most diagnostic part of this result. Twenty-one of them are date, time, and numeric-offset markers; sixteen are numeric-literal markers, mostly float overflow and malformed radix forms; seven concern table or key redefinition and unterminated composite constructs; and the remaining seven are isolated boolean, array-comment, multiline-string and literal-string cases. The pattern is consistent: Stage 2 handles a local syntax error well, and struggles where the intended marker is easily shadowed by an earlier parse error, or where the input must first navigate a long chain of numeric or temporal subparsers before it can reach the labelled branch. Note that we did not fill the misses with hand-written tests. The accepted witnesses *are* the suite, and the artifact records the marker-level ledger including every miss by name.

RQ4: is the solver doing real work? This question was put to us in review, and it deserves a measured answer rather than an argument. The per-round histories were recorded

for 156 of the 182 markers.[3] Over those markers the model made 649 proposals that reached Z3 and got a verdict or a timeout. Of these, 105 were accepted, 537 were rejected as `unsat`, and 7 produced no verdict within budget; a further 64 proposals were duplicates of already-rejected candidates and were dropped before Z3, and on 5 occasions the model returned no parsable candidate at all.

So roughly five out of every six proposals did not reach the marker they were labelled with. Reading the rejections, the usual cause is shadowing: the proposed document contains an earlier violation, so the parser stops before the intended branch. This matters for the obvious objection, which is that a language model could produce such a suite on its own. It could produce the *documents*; what it could not produce is the labelling. Without the query there is nothing against which to check the claim that a given document exercises a given rule, and an unaudited suite of this kind would have carried several hundred tests filed under the wrong obligation. It is also worth being clear about the converse, which the reviewers were right to press on: on a fully pinned input Z3 is *deciding* rather than searching, so on the TOML suite its generative role is empty. The division of labour is that the model carries generation, while the solver carries acceptance and, at Stage 1, unreachability.

RQ2: acceptance without inspection. No witness in either suite was accepted on the strength of model output. Every one was accepted because Z3 decided the marker query, which by the strengthening lemma of §5 is a statement about the query the author's marker generated, and every one was then re-executed through the concrete semantics and required to fire the same named marker. Note that this is the reason there is no triage step in the workflow: an accepted test is not a candidate awaiting human confirmation.

RQ5: what the suite found. We ran the 131 generated documents against four implementations: the Rust `toml` crate `1.1.3+spec-1.1.0`, CPython `tomllib` from CPython 3.14.4, BurntSushi `toml` v1.6.0 for Go, and `smol-toml` 1.7.1 for Node. Every document in the suite is one the RuSMT oracle rejects, so a divergence is an implementation that accepts it. Table 2 gives the counts; there were 15 divergences over 131×4 runs. All four parsers agreed with the oracle on 120 of the 131 documents, so the divergences concentrate in 11 markers.

We triaged all 15 against the standard, and the split is not the one we expected. Three are cases the standard settles *against the implementation.* `smol-toml` accepts `[[a]`, the running example of this paper, where an array-of-tables header requires a name in double brackets. BurntSushi accepts `a = { b = 1 }\n a.c = 2`, although TOML states that inline tables are fully self-contained and that keys and sub-tables cannot be added outside the braces; it also accepts `a.b = 1\n[a]`, where a `[table]` header redefines a table that a dotted key has already created implicitly, which TOML likewise disallows. These three are being reported upstream.

Table 2. Implementation agreement with the RuSMT oracle on the generated TOML suite. Every suite input is rejected by the oracle, so a divergence is an acceptance.

Implementation	Agreement	Divergences
Rust `toml 1.1.3+spec-1.1.0`	130/131	1
CPython `tomllib` (3.14.4)	123/131	8
BurntSushi `toml` v1.6.0 (Go)	128/131	3
Node `smol-toml` 1.7.1	128/131	3

The remaining twelve go the other way: our own reference is *stricter than the standard requires.* Six are CPython `tomllib` accepting integer literals outside the 64-bit signed range, namely `+9223372036854775808`, `-9223372036854775809`, `18446744073709551616`, `0x8000000000000000`, and the corresponding oversized binary and octal forms. Our reference rejects them, but TOML only *recommends* that at least 64-bit signed integers be accepted and handled losslessly, and requires an error only when an integer *cannot* be represented losslessly. CPython's arbitrary-precision integers represent all six exactly, so on that reading CPython is within its rights and we are the strict one. We should say that the opposite reading is not unreasonable: one can take a TOML integer simply *to be* 64-bit, so that anything wider is invalid whatever the host can represent. The wording has been discussed upstream, and it is precisely because we cannot settle it from the text that we report these six as triage cases rather than filing them. The other six are the two markers `float_exp_only_exp_overflow_i32` and `float_exp_only_exp_underflow_i32`, on inputs such as `x=1e2147483648`, which every one of the four parsers accepts in at least one direction. Here TOML leaves precision to the implementation and mandates nothing about exponent overflow, and our marker names carry `i32` only because the reference bounds the exponent by a 32-bit integer, which is an artifact of how we wrote the float parser rather than anything we can point to in the standard.

We want to dwell on that second group, because it is the outcome we did not design for and, on reflection, the more useful one. A divergence is a candidate bug *in either direction*: the reference is our own formalisation and not a normative document, so the suite tests the reference against the standard just as much as it tests the implementations against the reference. Twelve of the fifteen turned out to be the reference at fault rather than the parser. Each arrived as a minimal document labelled with the rule at issue, which is what made the triage tractable at all; the same twelve would

[3]The other 26 markers were covered by an earlier sequential pass that predates the per-marker sweep driver. It preserved the accepted witnesses but not the per-round outcomes, and we did not re-run those markers, since doing so would have spent model calls to recover bookkeeping we already know the result of. They are counted in the 131-input suite and excluded from this audit.

be nearly invisible in a hand-written suite, because a person writing tests from a reference tends to encode the reference's assumptions along with them.

The result we are least able to defend. For completeness we should say which of these numbers is the softest. It is not the coverage or the divergence count, both of which are mechanically reproducible from the recorded run; the conformance diff, re-run after the fact, reproduced byte for byte. It is the proposal audit, which covers 156 of 182 markers for the bookkeeping reason given above, and which describes one run of one nondeterministic model on one machine.

Artifact. The artifact contains the two reference semantics, the transpiler and backend, the four implementation runners with their pinned versions, and the commands that produced every number above.[4] It also archives the parts of the run that a re-execution cannot recover, namely the generated suites, the marker-level ledger with all 51 misses, the differential result, and the recorded proposer exchanges keyed by prompt hash. What is deterministic is regenerated rather than archived: the emitted per-marker queries involve neither the model nor the solver and render in seconds from the reference semantics, and the differential result is a pure function of the suite.

8 Related work

Solver-aided executable semantics. Rosette [23, 24] is the closest solver-aided host: a symbolic virtual machine compiles a language subset to constraints while concrete evaluation strips away ordinary host code. RuSMT is narrower and more conformance-oriented, in that the user writes a reference interpreter, marks specification obligations, and receives tests that were accepted by a solver. SemGuS [9] synthesizes from syntax and semantics encoded as constrained Horn clauses, whereas RuSMT derives its formulas from ordinary recursive functions. Synantic [11] runs in the opposite direction, synthesizing a formal semantics from a black-box interpreter, which is complementary to our source-to-SMT route.

Specification-derived testing. Executable-semantics frameworks such as K [18], PLT Redex [7], and Ott/Lem [14, 22] derive tool suites from rewrite or inference-rule specifications. RuSMT derives a narrower tool from direct-style Rust, namely marker-directed conformance tests. JEST [15] is the closest in purpose, using a mechanized ECMAScript specification as oracle, synthesizer, and conformance tester; our distinction is the acceptance discipline on author-named markers and the explicit sound-but-incomplete split that the TOML case study exposes. Coverage- and grammar-driven generators such as Csmith [29] remain the application domain rather than the technique.

Language models behind a solver. We claim no novelty for the propose-then-check pattern; it is CEGIS with a language model in the generator position. Lemur [28] uses LLM-proposed verification structure, and AquaForte [12] has a model propose SMT instantiations that the solver then rechecks. Test-generation portfolios such as CodaMOSA [10] and Cottontail [25] similarly call a model only once conventional generation stalls. Our modest difference is where the checked obligation lives: the same executable semantics is both oracle and synthesis source, and the accepted obligation is a stable named marker rather than generic coverage.

9 Limitations and threats to validity

Scope and subset. The DSL excludes loops, mutation, and references. The price buys a clean SMT translation and a specification that mirrors textbook big-step semantics, and it is mechanically enforced, but it is a real restriction on what can be written. More importantly, a marker covers only an error condition the author actually *wrote*. A violation the standard implies but our reference omits is simply not marked, so the TOML suite is marker-complete relative to our formalisation and not to TOML 1.1.0.

One witness per marker. The reported run used `RUSMT_WITNESSES=1`, which is thin, and a reviewer was right to say so. A single witness may be an input every implementation already handles, and it certainly under-tests a rule with several distinct ways of being violated. The mechanism for more is present, since additional witnesses cost one blocking assertion and a re-solve each, and the framework's own default is three; we set it to one to bound the number of model calls across 182 markers. We have not measured what a larger setting buys, and we do not want to guess.

The authoring burden. The artifact is write-once and use-many: one source is the oracle, the synthesis source, and the replay oracle, so the effort amortizes over a reusable specification rather than a throwaway generator. Nevertheless the burden is lowered rather than removed. A human still has to choose a DSL-expressible subset, place the markers, and own the result. The approach therefore pays off when a reusable reference semantics is wanted anyway, and it is poor economics for a one-off test.

Authoring the semantics with a model. The framework contains a gated authoring mode in which a proposer drafts a reference semantics from a prose specification and each draft must pass the mechanical and behavioural gates before a human sees it. We deliberately report no numbers for it. Measuring whether a model writes a better interpreter directly in Rust or through this DSL is a separate study with

[4] https://github.com/mehrad31415/rusmt

its own confounds, and asserting an outcome we have not measured is exactly the failure mode this revision was meant to remove.

The solver frontier. The dominant limitation is solver capability on deeply recursive, multi-theory queries, and we would rather characterize it than report it as a wall. Z3 decides a *determined* input on the TOML parser quickly and returns nothing useful for a symbolic one at any budget we tried, because the transpiled parsers are macros expanded before branch conditions can fold. That diagnosis is what makes Stage 2 principled rather than opportunistic, but it does not make the solver complete, and 51 markers remain uncovered. A natural next step is Z3's user-propagator API, which offers callbacks at decision points the solver controls and needs no vendored patch, so that a proposer could inject solver-rechecked lemmas [12, 28], or, more ambitiously, so that the concrete reference semantics could be registered as a propagated theory and a candidate replayed *inside* the search, with a rejection returning a conflict clause instead of merely ending a round. Notably this needs no second encoder, since `Z3_solver_from_string` loads the backend's own emitted SMT-LIB into an in-memory solver.

We should also record a negative result about generalization, since it is the first thing one wants from a witness. We tried asserting a *pattern* and checking that its negation is `unsat`, which would upgrade a single witness to a proved family. A query with even one free character returned no verdict in 300 s. This is future work at best, and on the present evidence we cannot claim that any witness generalizes.

Threats to validity. Read against the standard empirical-software-engineering taxonomy [19, 27], the following are the main threats. *Construct:* a marker measures an error the reference author *wrote*, not the standard's true error space, and the per-construct correspondence of §5 is assumed rather than proven, so "the lift is faithful" is a trusted premise with documented exceptions and not a result. *Internal:* acceptance is by solver verdict on a *named* marker whose id coincides on both sides by construction, and replay requires that same target to fire, which excludes "some-marker" confounds and the spurious depth-bounded models of §4; the array-free re-encoding was controlled by checking byte-identical concrete output across both encodings on every test, which is regression-preserving rather than a proof of equivalence. *External:* two case studies, one machine, one solver version, one model, and no cross-machine or cross-model claim. *Conclusion:* the TOML run is a single recorded run of a nondeterministic model, and Z3's heuristics make latencies run-dependent, so we report the qualitative regime and content-address every query and prompt.

10 Conclusion

RuSMT lets an author write a language's reference semantics once, as an ordinary recursive Rust program that is at the same time the conformance oracle and the source of the tests. Z3 alone closes the loop for IMP. On the larger TOML parser it cannot, for a reason we were able to pin down, and a model proposes concrete documents from the emitted SMT-LIB while Z3 decides whether each one reaches the named marker. The outcome is a generated TOML conformance suite covering 131 of 182 markers, which exposed 15 accept/reject divergences across four production parsers. Three of those turned out to be parser bugs and twelve to be places where our own reference is stricter than TOML requires, which we did not anticipate and now think is the more useful half of the result: the same suite that tests implementations against the reference tests the reference against the standard. The suite is not complete and the misses are reported by name; what we do claim is that no test in it was accepted because a person or a model said it was right. In other words, the specification becomes a living artifact: when it changes, the conformance suite is regenerated rather than re-authored.

Acknowledgments

We thank the SPECops 2026 reviewers, whose comments on the submitted version prompted the implementation comparison of §7, the removal of a fixed candidate table in favour of live proposer calls, and the reframing of §5.

References

[1] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-Guided Synthesis. In *2013 Formal Methods in Computer-Aided Design (FMCAD 2013)*. IEEE, 1–8.

[2] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2017. *The SMT-LIB Standard: Version 2.6.* Technical Report. Department of Computer Science, The University of Iowa. Available at https://smt-lib.org.

[3] Armin Biere, Alessandro Cimatti, Edmund M. Clarke, and Yunshan Zhu. 1999. Symbolic Model Checking without BDDs. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS) (LNCS, Vol. 1579)*. Springer, 193–207. doi:10.1007/3-540-49059-0_14

[4] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. 2014. Using Frankencerts for Automated Adversarial Testing of Certificate Validation in SSL/TLS Implementations. In *2014 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 114–129. doi:10.1109/SP.2014.15

[5] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2008) (Lecture Notes in Computer Science, Vol. 4963)*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24

[6] Ecma International TC39. [n. d.]. Test262: ECMAScript Conformance Test Suite. https://github.com/tc39/test262. Accessed 2026-06-20.

[7] Matthias Felleisen, Robert Bruce Findler, and Matthew Flatt. 2009. *Semantics Engineering with PLT Redex.* MIT Press.

[8] Haryadi S. Gunawi, Cindy Rubio-González, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Ben Liblit. 2008. EIO: Error Handling

is Occasionally Correct. In *6th USENIX Conference on File and Storage Technologies (FAST)*. USENIX Association, 207–222.

[9] Jinwoo Kim, Qinheping Hu, Loris D'Antoni, and Thomas W. Reps. 2021. Semantics-Guided Synthesis. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–32. doi:10.1145/3434311

[10] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMOSA: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *45th International Conference on Software Engineering (ICSE 2023)*. IEEE. doi:10.1109/ICSE48619.2023.00085

[11] Jiangyi Liu, Charlie Murphy, Anvay Grover, Keith J. C. Johnson, Thomas W. Reps, and Loris D'Antoni. 2024. Synthesizing Formal Semantics from Executable Interpreters. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024). doi:10.1145/3689724 Introduces the Synantic tool.

[12] Kunhang Lv, Yuhang Dong, Rui Han, Fuqi Jia, Feifei Ma, and Jian Zhang. 2026. LLM-Guided Quantified SMT Solving over Uninterpreted Functions. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 17 (2026), 14304–14312. doi:10.1609/aaai.v40i17.38445 Introduces the AquaForte framework.

[13] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.

[14] Dominic P. Mulligan, Scott Owens, Kathryn E. Gray, Tom Ridge, and Peter Sewell. 2014. Lem: Reusable Engineering of Real-World Semantics. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP 2014)*. ACM, 175–188. doi:10.1145/2628136.2628143

[15] Jihyeok Park, Seungmin An, Dongjun Youn, Gyeongwon Kim, and Sukyoung Ryu. 2021. JEST: N+1-version Differential Testing of Both JavaScript Engines and Specification. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE 2021)*. IEEE, 13–24. doi:10.1109/ICSE43902.2021.00015

[16] Benjamin C. Pierce. 2002. *Types and Programming Languages*. MIT Press.

[17] Tom Preston-Werner et al. 2025. TOML: Tom's Obvious, Minimal Language, Version 1.1.0. https://toml.io/en/v1.1.0. Language specification.

[18] Grigore Roşu and Traian Florin Şerbănuţă. 2010. An Overview of the K Semantic Framework. *The Journal of Logic and Algebraic Programming* 79, 6 (2010), 397–434. doi:10.1016/j.jlap.2010.03.012

[19] Per Runeson and Martin Höst. 2009. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering* 14, 2 (2009), 131–164. doi:10.1007/s10664-008-9102-8

[20] Len Sassaman, Meredith L. Patterson, Sergey Bratus, and Michael E. Locasto. 2013. Security Applications of Formal Language Theory. *IEEE Systems Journal* 7, 3 (2013), 489–500. doi:10.1109/JSYST.2012.2222000

[21] Nicolas Seriot. 2016. Parsing JSON is a Minefield. http://seriot.ch/parsing_json.php. Accessed 2026-06-20.

[22] Peter Sewell, Francesco Zappa Nardelli, Scott Owens, Gilles Peskine, Tom Ridge, Susmit Sarkar, and Rok Strniša. 2007. Ott: Effective Tool Support for the Working Semanticist. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*. ACM, 1–12. doi:10.1145/1291151.1291155

[23] Emina Torlak and Rastislav Bodík. 2013. Growing Solver-Aided Languages with Rosette. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software (Onward! 2013)*. ACM, 135–152. doi:10.1145/2509578.2509586

[24] Emina Torlak and Rastislav Bodík. 2014. A Lightweight Symbolic Virtual Machine for Solver-Aided Host Languages. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2014)*. ACM, 530–541. doi:10.1145/2594291.2594340

[25] Haoxin Tu, Seongmin Lee, Yuxian Li, Peng Chen, Lingxiao Jiang, and Marcel Böhme. 2025. Cottontail: Large Language Model-Driven Concolic Execution for Highly Structured Test Input Generation. *arXiv preprint arXiv:2504.17542* (2025). arXiv:2504.17542 To appear, IEEE Symposium on Security and Privacy (S&P) 2026.

[26] Glynn Winskel. 1993. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press.

[27] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in Software Engineering*. Springer. doi:10.1007/978-3-642-29044-2

[28] Haoze Wu, Clark Barrett, and Nina Narodytska. 2024. Lemur: Integrating Large Language Models in Automated Program Verification. In *International Conference on Learning Representations (ICLR 2024)*. arXiv:2310.04870

[29] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2011)*. ACM, 283–294. doi:10.1145/1993498.1993532

[30] Ding Yuan, Yu Luo, Xin Zhuang, Guilherme Renna Rodrigues, Xu Zhao, Yongle Zhang, Pranay U. Jain, and Michael Stumm. 2014. Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 249–265.